Agent Team Analysis

Agent Team Analysis

Table of Contents

1. System Positioning and Overall Architecture
   - 1.1 System Positioning
   - 1.2 Overall Architecture Diagram
   - 1.3 Core Design Principles
   - 1.4 Core Design Philosophy
2. Directory Structure and File Index
3. Core Data Types in Detail
   - 3.1 TeamFile —— Team Configuration File
   - 3.2 TeammateContext —— Runtime Context (AsyncLocalStorage)
   - 3.3 InProcessTeammateTaskState —— In-Process Member Task State
   - 3.4 LocalAgentTaskState —— Local Agent Task State
   - 3.5 TeammateMessage —— Mailbox Message
   - 3.6 SwarmPermissionRequest —— Permission Request (pending/resolved directories)
   - 3.7 TeamTask —— Shared Task Entry
4. Team Creation and Management
   - 4.1 TeamCreateTool Complete Flow
   - 4.2 Adding Team Members
   - 4.3 Team Destruction (Shutdown Protocol)
5. Agent Lifecycle Management
   - 5.1 Lifecycle State Machine
   - 5.2 runAsyncAgentLifecycle Complete Implementation
6. In-Process Execution Engine
   - 6.1 Core Configuration
   - 6.2 Complete Message Loop
   - 6.2.1 Deep Analysis: Dual-Layer AbortController Design
   - 6.2.2 Deep Analysis: Content Replacement State Historical Memory
   - 6.3 waitForNextPromptOrShutdown Priority Algorithm
   - 6.4 Permission Bridge Implementation
   - 6.4.1 Deep Analysis: Dual-Channel Permission Bridge Design and Anti-Escalation Mechanism
7. Inter-Process Communication (IPC) —— Mailbox System
   - 7.1 Mailbox File Format
   - 7.2 File Lock Implementation
   - 7.3 Message Reading Strategy
   - 7.4 Message Serialization Format
8. Permission Management System
   - 8.1 Three-Layer Permission Model
   - 8.2 pending/resolved Directory Mechanism

- 8.3 Permission Cleanup Mechanism

9. Execution Backend Abstraction Layer
    - 9.1 PaneBackend Interface
    - 9.2 Backend Auto-Detection Algorithm
    - 9.3 Tmux Backend Key Commands
    - 9.4 In-Process Backend
10. Team Task List and Distributed Task Claiming
    - 10.1 Task Directory Structure
    - 10.2 Task Claiming Algorithm (Optimistic Lock + File Lock)
    - 10.2.1 Deep Analysis: Concurrency Safety and DAG Dependency Detection
11. Agent Toolchain and Sub-Agent Dispatch
    - 11.1 AgentTool Parameter Schema
    - 11.2 Tool Filter
    - 11.3 Agent Result Integration
12. Coordinator Mode
    - 12.1 Coordinator Role Definition
    - 12.2 Coordinator Task Decomposition Algorithm
13. State Management (AppState)
    - 13.1 Team-Related AppState Structure
    - 13.2 Team Memory Synchronization
    - 13.3 Deep Analysis: Three-Layer Identity Recognition and Zero-Overhead Isolation
14. Key Constants Quick Reference
15. Key Algorithm Detailed Flows
    - 15.1 Teammate Complete Startup Flow
    - 15.2 Permission Request-Response Complete Sequence
    - 15.3 Shared Task Claiming Sequence
16. Summary and Design Evaluation
    - 16.1 Core Design Highlights
    - 16.2 Architectural Trade-offs
    - 16.3 Main Limitations
    - 16.4 Deep Design Philosophy Analysis
    - 16.5 Extension Suggestions
    - 16.6 Horizontal Comparison with Mainstream Multi-Agent Frameworks

---

1. System Positioning and Overall Architecture

1.1 System Positioning

This is a **Multi-Agent Swarm Orchestration System** running within the Claude Code CLI. Its core responsibility is: **decomposing a complex user request into multiple parallel subtasks, completed collaboratively by several Agents (team members), with the Team Lead (coordinator) integrating the final results.**

Technical positioning:

- **Leader-Worker Pattern**: Team Lead handles task assignment, permission approval, and result integration

- **Swarm Architecture**: Multiple agents work in parallel, **coordinating through message passing**
- **Supports Two Execution Modes**: In-process isolation and process-level isolation (tmux/iTerm2 Pane)
- **Zero External Dependency Message Queue**: IPC implemented via filesystem + file locks

> **Note**
>
> Claude Code's **Agent Team mode** is **disabled by default** and requires explicit user activation. Two main ways to enable:
>
> - Manual command: Run `claude --team` or use the `/team` command in the interactive terminal
> - Configuration file: Set `"team": { "enabled": true }` in `~/.claude/settings.json`
>
> The core scenario for Team mode: when a task is decomposed into multiple parallel independent subtasks, the main Agent automatically schedules multiple sub-agents to work collaboratively, each handling different parts of the problem from different angles, with results finally aggregated. Simple tasks do not automatically trigger this; the complexity threshold is judged internally by the framework.

1.2 Overall Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                           User (User)                                    │
│                  Inputs task in Claude Code CLI                            │
└──────────────────────────────────┬──────────────────────────────────────┘
                                   │ /team-create, /agent, /send-message
                                   ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     Team Lead (Leader / Coordinator)                     │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  TeamContext (AppState)                                          │   │
│  │   • teamName, leadAgentId                                        │   │
│  │   • members[] (teammate identity list)                           │   │
│  │   • inbox (Mailbox) ←── Polls for new messages                   │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐      │
│  │  TeamCreateTool  │  │   AgentTool      │  │ SendMessageTool│  │
│  │   (Create Team)   │  │  (Dispatch Sub-Agent)│  │   (Send Message)│  │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘      │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  ToolUseConfirm UI (Permission Approval Dialog)                  │   │
│  │   • Sub-Agent requests permission → Popup → User approves → Result written back to
Mailbox       │  │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
           │                                      │
           │                                      │
    ┌──────▼──────┐                     ┌──────────▼──────────┐
    │ In-Process  │                     │  tmux / iTerm2    │
    │ Teammate A  │                     │  Pane (Agent B)    │
    │             │                     │                    │
    │ AsyncLocal   │                     │ Independent OS process │
```

```
    │ Storage ctx  │                    │ Independent tmux session│
    └──────────────┘                    └────────────────────────┘
          │                                       │
          └──────────────────────────────────────┘
                          │
                          │ Mailbox IPC (JSON + File Lock)
                          ▼
        ┌──────────────────────────────────────┐
        │  ~/.claude/teams/{team}/inboxes/     │
        │  teammate_a.json   (message array)   │
        │  teammate_b.json                      │
        │  leader.json       (Team Lead inbox) │
        └──────────────────────────────────────┘
                          │
                          ▼
        ┌──────────────────────────────────────┐
        │  ~/.claude/teams/{team}/permissions/ │
        │  pending/{requestId}.json            │
        │  resolved/{requestId}.json           │
        └──────────────────────────────────────┘
```

Note

- Two running modes for sub-agents:
  - In-process: Same process, context isolated via AsyncLocalStorage, permission approval goes directly through Team Lead's ToolUseConfirm UI
  - Tmux pane: Independent OS process/independent tmux session, communication via Mailbox IPC using file locks + JSON files
- Mailbox path: `~/.claude/teams/{team}/inboxes/{agent_name}.json`, message format is JSON array, supports concurrent writes with file locks

1.3 Core Design Principles

Principle	Implementation
Leader Centralization	Team Lead controls all permission approvals, task assignments, and result integration
Message-Driven	Team members communicate asynchronously via Mailbox message polling
Permission Bridging	Sub-Agent permission requests are **forwarded to Leader UI for user approval**
Graceful Shutdown	ShutdownRequest/ShutdownApproved bidirectional confirmation protocol
Pluggable Backend	Unified PaneBackend interface supports tmux / iTerm2 / In-Process
Context Isolation	AsyncLocalStorage **maintains independent context for each In-Process member**

1.4 Core Design Philosophy

Why File Polling Instead of Event-Driven?

The surface reason is "simple, no external dependencies," but the deeper reason lies in **Node.js's fs module constraints**. Claude Code runs in Electron, where the UI thread is the event loop, and synchronous file operations are forbidden. The async version of `proper-lockfile` lacks atomic "check-modify-write-back" semantics, so **polling + retry** is used to compensate.

A comment in the code (`teammateMailbox.ts` lines 31-34) reveals this:

> "The sync lockSync API blocked the event loop; the async API needs explicit retries to achieve the same serialization semantics."

Alternative Assessment:

- Unix Domain Socket / Redis Pub/Sub: Introduces external dependencies, conflicts with Claude Code's offline-first design
- WebSocket: Requires a message server, beyond necessary scope
- **Conclusion**: Implement reliable cross-process communication with minimal external dependencies

Read-Write Separation Lock Strategy

```
// Read: No lock (accepts eventual consistency)
export async function readMailbox(...) { return messages }

// Write: Requires lock
export async function writeToMailbox(...) {
  release = await lockfile.lock(inboxPath, LOCK_OPTIONS)
  // Re-read to get latest state (solves TOCTOU problem)
  const messages = await readMailbox(...)
  messages.push(newMessage)
  await writeFile(...)
}
```

Design Intent:

- **Lock-free reads**: Improves read performance, accepts brief inconsistency (multiple readers may briefly see different message lists)
- **Write with lock**: **Ensures atomicity of append operations**, prevents message loss from concurrent writes
- **Re-read on write**: Critical! Solves the "Time-of-check to time-of-use" (TOCTOU) problem — between acquiring the lock and the write operation, another process may have modified the file

> **Supplementary: Why does markMessageAsRead also use a lock, instead of read-modify-write semantics?**
>
> In the source code (`teammateMailbox.ts` lines 196-271), `markMessageAsReadByIndex` uses file lock + read-modify-write pattern — consistent with `writeToMailbox`. But unlike normal message writes, `markMessageAsRead` is **idempotent operation concurrent protection**:
>
> Scenario: Two consumers (useInboxPoller and getTeammateMailboxAttachments) read the same file simultaneously. Consumer A marks message M1 as read, Consumer B also wants to mark M1 as read.
>
> Without locking:
>
> ```
> Consumer A: Read file → M1.read=false → Modify in memory → Write back (M1.read=true)
> Consumer B: Read file (Consumer A hasn't written back yet!) → M1.read=false → Modify in memory → Write back (M1.read=true)  // Overwrites Consumer A's result
> // Result: M1 is marked as read, which is correct — it's already idempotent
> ```
>
> In this scenario, "no lock" seems fine (because writing the same value repeatedly is harmless). The real problem is:

```
Consumer A: Read file → Decide to mark M1
Consumer B: Deletes M2 (received deletion instruction from another entry) → Write back
Consumer A: On write back, M1's index has shifted to where M2 was → Wrongly marks another message
```

Additionally, `markMessageAsReadByIndex`'s implementation (`teammateMailbox.ts` lines 238-242) has a defensive check: if index is out of bounds or the message is already read, it returns directly without writing. This further reduces the actual impact of concurrency issues — even if there's a TOCTOU, what's overwritten is just a state that's already `read=true`. This is a **defensive programming + idempotency guarantee** double safety net.

Lock Retry Exponential Backoff Design

```
const LOCK_OPTIONS = {
  retries: { retries: 10, minTimeout: 5, maxTimeout: 100 },
  // Backoff sequence: 5, 10, 20, 40, 80, 100, 100, 100, ...
}
```

10 retries + exponential backoff (max 100ms) is designed for approximately **10 concurrent agents**

---

2. Directory Structure and File Index

```
restored-src/src/
│
├── coordinator/
│   └── coordinatorMode.ts          # Coordinator system prompt template
│
├── tasks/
│   ├── types.ts                    # 7 TaskState union type definitions
│   ├── LocalAgentTask/
│   │   └── LocalAgentTask.tsx      # Local Agent task state management and rendering
│   └── InProcessTeammateTask/
│       ├── types.ts                # InProcessTeammateTaskState complete types
│       └── InProcessTeammateTask.tsx # Task lifecycle management
│
├── tools/
│   ├── AgentTool/
│   │   ├── AgentTool.ts            # Tool definition (parameter schema + call)
│   │   └── agentToolUtils.ts        # Core logic: tool filtering, lifecycle management
│   ├── TeamCreateTool/
│   │   └── TeamCreateTool.ts       # Team creation tool
│   ├── TeamDeleteTool/
│   │   └── TeamDeleteTool.ts       # Team destruction tool
│   └── SendMessageTool/
│       └── SendMessageTool.ts      # Message sending (supports structured messages)
│
├── utils/
│   ├── teammate.ts                  # Team member identity management (getAgentId, etc.)
│   ├── teammateContext.ts          # AsyncLocalStorage context management
│   ├── teammateMailbox.ts          # File mailbox IPC (1183 lines, core IPC)
│   ├── teamDiscovery.ts            # Team discovery
```

```
│   ├── tasks.ts                    # Task list CRUD + file lock (863 lines)
│   └── swarm/
│       ├── teamHelpers.ts          # TeamFile CRUD + AppState helpers
│       ├── inProcessRunner.ts      # In-Process execution main loop (1552 lines, core)
│       ├── spawnInProcess.ts       # In-process spawn logic (328 lines)
│       ├── permissionSync.ts       # Permission sync pending/resolved directories (928
lines)
│       ├── constants.ts            # Swarm constants
│       ├── addMemberToTeam.ts      # Add member to team
│       ├── removeMemberFromTeam.ts # Remove member from team
│       └── backends/
│           ├── types.ts            # PaneBackend interface definition
│           ├── registry.ts         # Backend auto-detection registry
│           ├── TmuxBackend.ts      # tmux backend implementation
│           ├── ITermBackend.ts     # iTerm2 backend implementation
│           ├── InProcessBackend.ts # In-process backend
│           └── detection.ts        # tmux/iTerm2 detection logic
│
└── services/
    ├── teamMemorySync/             # Team memory synchronization service
    └── AgentSummary/               # Agent long message summary service
```

---

3. Core Data Types in Detail

3.1 TeamFile —— Team Configuration File

Definition: `src/utils/swarm/teamHelpers.ts` (line 683)

TeamFile is the central configuration hub of the team, persisted at `~/.claude/teams/{team}/config.json`:

```ts
export type TeamFile = {
  // Basic Info
  name: string                      // Team name (globally unique)
  description?: string              // Team description (optional)
  createdAt: number                // Creation timestamp (Date.now())

  // Leader Info
  leadAgentId: string              // Leader Agent ID (format: "team-lead@my-team")
  leadSessionId?: string           // Leader session ID

  // Hidden panes (tmux hidden windows)
  hiddenPaneIds?: string[]

  // Team allowed paths (permission boundary)
  teamAllowedPaths?: TeamAllowedPath[]

  // Member list (core)
  members: Array<{
    // Identity
    agentId: string                // Member unique ID ("researcher@my-team")
    name: string                   // Display name (without team suffix)

    // Agent configuration
```

```
    agentType?: string              // Agent type (e.g., "researcher")
    model?: string                  // Model to use
    prompt?: string                 // Initial prompt

    // UI
    color?: string                 // UI color (for distinguishing members)

    // Permission
    planModeRequired?: boolean      // Whether plan submission is required first

    // Join time
    joinedAt: number                // Join timestamp

    // Execution environment
    tmuxPaneId: string              // tmux pane ID (empty string means unassigned)
    cwd: string                    // Working directory
    worktreePath?: string           // Git worktree path (isolated branch)

    // State
    sessionId?: string             // Member session ID
    subscriptions: string[]         // Subscribed topic list (for pub/sub)
    backendType?: BackendType      // Execution backend: 'tmux' | 'iterm2' | 'in-process'
    isActive?: boolean              // Whether online
    mode?: PermissionMode           // Current permission mode
  }>
}

// Team allowed path entry
export type TeamAllowedPath = {
  path: string                     // Directory path (absolute)
  toolName: string                 // Applicable tool name (e.g., "Read", "Write")
  addedBy: string                  // Member name who added this rule
  addedAt: number                  // Added timestamp
}
```

Note

Agent Team has two independent permission control mechanisms, both of which can be **dynamically configured by the Leader, not statically preset**

- First layer: mode (permission mode) — member level. Stored in config.json member entries, can be dynamically modified at any time via Leader's TeamsDialog
- Second layer: teamAllowedPaths (allowed paths) — global level. All Workers share this list; Leader can add/remove entries. When Workers access paths in this list, they're automatically allowed without requesting Leader permission

> **Design Essential Insight**: The combination of these two permission layers forms a **Dynamic Access Matrix** — rows are members, columns are tools/paths, each cell in the matrix is "whether approval is needed". `mode` controls row-level default behavior (entire row is ask or bypass), while `teamAllowedPaths` punches holes in specific paths (partial bypass). This model is more flexible than simple "all allow/all deny" and more concise than configuring each member-tool combination individually.

> Notably, `planModeRequired` is a special value of `mode` — it doesn't just change whether a popup appears, it changes the Agent's **behavior protocol**: in plan mode, the Agent must first output a plan and wait for approval before executing writes. This overlays a **workflow constraint** on top of permissions, extending the question from "who can do what" to "under what conditions can they do it."

3.2 TeammateContext —— Runtime Context (AsyncLocalStorage)

Definition: `src/utils/teammateContext.ts`

Each In-Process member maintains its own context at runtime via AsyncLocalStorage:

```
export type TeammateContext = {
  // Identity (same as TeammateIdentity)
  agentId: string                  // Full ID: "researcher@my-team"
  agentName: string                // Display name
  teamName: string                 // Team name

  // UI
  color?: string                   // Color

  // Permission
  planModeRequired: boolean         // Whether plan approval is required

  // Leader info
  parentSessionId: string          // Team Lead's session ID (for permission bridging)

  // Type discriminator (TypeScript union type differentiation)
  isInProcess: true

  // Abort controller
  abortController: AbortController
}

// Key functions
export function runWithTeammateContext<T>(
  ctx: TeammateContext,
  fn: () => T
): T

export function getTeammateContext(): TeammateContext | null

export function createTeammateContext(
  identity: TeammateIdentity,
  abortController: AbortController
): TeammateContext
```

3.3 InProcessTeammateTaskState —— In-Process Member Task State

Definition: `src/tasks/InProcessTeammateTask/types.ts`

This is the core state object for In-Process members:

```typescript
export type InProcessTeammateTaskState = TaskStateBase & {
  type: 'in_process_teammate'

  // Identity
  identity: TeammateIdentity

  // Execution config
  prompt: string
  model?: string
  selectedAgent?: AgentDefinition

  // Abort controllers (lifecycle)
  abortController?: AbortController
  // Current work round abort controller (more granular)
  currentWorkAbortController?: AbortController
  unregisterCleanup?: () => void

  // Permission
  awaitingPlanApproval: boolean   // Whether waiting for plan approval
  permissionMode: PermissionMode  // Current permission mode

  // Result
  error?: string
  result?: AgentToolResult
  progress?: AgentProgress        // { toolUseCount, tokenCount, lastActivity, ... }

  // Message history
  messages?: Message[]
  inProgressToolUseIDs?: Set<string>  // IDs of tool calls currently executing

  // Pending injected messages (from SendMessageTool)
  pendingUserMessages: string[]

  // UI display
  spinnerVerb?: string            // Spinner animation verb (e.g., "Thinking")
  pastTenseVerb?: string          // Past tense verb (e.g., "Thought")

  // State flags
  isIdle: boolean                 // Whether idle (waiting for new task)
  shutdownRequested: boolean     // Whether shutdown request received

  // Idle callbacks
  onIdleCallbacks?: Array<() => void>

  // Progress
  lastReportedToolCount: number   // Last reported tool call count
  lastReportedTokenCount: number  // Last reported token count
}

// Message count cap (prevent memory bloat)
export const TEAMMATE_MESSAGES_UI_CAP = 50
```

```
// Auto-truncate to cap when appending messages
export function appendCappedMessage<T>(prev: readonly T[] | undefined, item: T): T[]
```

3.4 LocalAgentTaskState —— Local Agent Task State

Definition: `src/tasks/LocalAgentTask/LocalAgentTask.tsx`

```
export type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'

  // Identity and config
  agentId: string
  prompt: string
  selectedAgent?: AgentDefinition
  agentType: string
  model?: string

  // Abort
  abortController?: AbortController
  unregisterCleanup?: () => void

  // Result
  error?: string
  result?: AgentToolResult
  progress?: AgentProgress

  // Messages
  retrieved: boolean              // Whether loaded from disk
  messages?: Message[]
  lastReportedToolCount: number
  lastReportedTokenCount: number

  // Execution control
  isBackgrounded: boolean          // Whether running in background (doesn't occupy main
loop)
  pendingMessages: string[]        // Messages queued by SendMessageTool

  // UI retention
  retain: boolean                  // Retain this task (don't auto-cleanup)
  diskLoaded: boolean              // Whether loaded from JSONL sidecar file

  // Pane visibility
  evictAfter?: number              // Pane visibility expiration timestamp
}

// Agent execution progress
export type AgentProgress = {
  toolUseCount: number            // Total tool call count
  tokenCount: number              // Total token count
  lastActivity?: ToolActivity     // Most recent tool activity
  recentActivities?: ToolActivity[] // Recent activity list (for UI)
  summary?: string                // AI-generated progress summary
}
```

3.5 TeammateMessage —— Mailbox Message

Definition: `src/utils/teammateMailbox.ts` (1183 lines)

```
// Base message (any message is converted to this format for storage)
export type TeammateMessage = {
  from: string                    // Sender name (without team suffix)
  text: string                    // Message content (JSON-serialized string)
  timestamp: string               // ISO format timestamp
  read: boolean                   // Read flag
  color?: string                  // Sender's color
  summary?: string                // AI-generated short summary (for UI display)
}

// ===== Idle Notification =====
export type IdleNotificationMessage = {
  type: 'idle_notification'
  from: string
  timestamp: string
  idleReason?: 'available' | 'interrupted' | 'failed'
  summary?: string
  completedTaskId?: string
  completedStatus?: 'resolved' | 'blocked' | 'failed'
  failureReason?: string
}

// ===== Permission Request =====
export type PermissionRequestMessage = {
  type: 'permission_request'
  request_id: string
  agent_id: string
  tool_name: string
  tool_use_id: string
  description: string             // Natural language description of tool call
  input: Record<string, unknown>   // Tool input parameters
  permission_suggestions: unknown[] // Permission suggestions
}

// ===== Permission Response =====
export type PermissionResponseMessage =
  | {
      type: 'permission_response'
      request_id: string
      subtype: 'success'
      response?: {
        updated_input?: Record<string, unknown>
        permission_updates?: unknown[]
      }
    }
  | {
      type: 'permission_response'
      request_id: string
      subtype: 'error'
```

```
      error: string
    }

// ===== Sandbox Network Permission =====
export type SandboxPermissionRequestMessage = {
  type: 'sandbox_permission_request'
  requestId: string
  workerId: string
  workerName: string
  workerColor?: string
  hostPattern: { host: string }
  createdAt: number
}

export type SandboxPermissionResponseMessage = {
  type: 'sandbox_permission_response'
  requestId: string
  host: string
  allow: boolean
  timestamp: string
}

// ===== Plan Approval =====
export type PlanApprovalRequestMessage = {
  type: 'plan_approval_request'
  from: string
  timestamp: string
  planFilePath: string
  planContent: string
  requestId: string
}

export type PlanApprovalResponseMessage = {
  type: 'plan_approval_response'
  requestId: string
  approved: boolean
  feedback?: string
  timestamp: string
  permissionMode?: string
}

// ===== Shutdown Request =====
export type ShutdownRequestMessage = {
  type: 'shutdown_request'
  requestId: string
  from: string
  reason?: string
  timestamp: string
}

export type ShutdownApprovedMessage = {
  type: 'shutdown_approved'
  requestId: string
```

```
  from: string
  timestamp: string
  paneId?: string
  backendType?: string
}

export type ShutdownRejectedMessage = {
  type: 'shutdown_rejected'
  requestId: string
  from: string
  reason: string
  timestamp: string
}

// ===== Task Assignment =====
export type TaskAssignmentMessage = {
  type: 'task_assignment'
  taskId: string
  subject: string
  description: string
  assignedBy: string
  timestamp: string
}

// ===== Mode Change =====
export type ModeSetRequestMessage = {
  type: 'mode_set_request'
  mode: string
  from: string
}
```

Core Design: Two-Layer Message Model

The entire message system consists of **two layers**:

```
First layer: TeammateMessage (Transport Layer)
  - from / text / timestamp / read / color / summary
  - This is the "envelope" of the message, stored in the Mailbox file
  - The text field is a JSON string containing the second layer content

Second layer: Protocol message types (Protocol Layer)
  - idle_notification / permission_request / shutdown_request, etc.
  - These are the "letter content", serialized/deserialized as JSON in the text field
```

Field Design Intent

Field	Purpose	Design Decision
`from`	Sender identifier	**Without team suffix**, stores `"researcher"` not `"researcher@my-team"`, reduces storage redundancy (team name is already in the path)
`text`	JSON string	**Bidirectional decision**: Serialized storage solves JSON object consistency issues in files (field ordering, referential transparency)
`timestamp`	ISO 8601	**Cross-timezone readable**, more debuggable than Unix timestamp

Field	Purpose	Design Decision
`read`	Read flag	**Single-reader lock**: Written as `false` by default, marked read by the **receiver** after reading
`color`	UI color	**Flat storage**: Stores `workerColor` directly in the message, avoids querying TeamFile (reduces IO)
`summary`	AI summary	**5-10 word preview**, reduces UI rendering overhead; generated by sender when creating the message

Concurrent Safety of the `read` Field

The `read` field follows a "who reads it marks it" pattern — **the receiver marks it as read after successfully processing it**. `writeToMailbox` always writes `read: false`.

> **Semantic Analysis**: The semantics of the `read` field is "**receiver knows about it**" not "**sender confirms receipt**". This means:
>
> - Sender writes `read: false` optimistically — assuming the other party doesn't know about this message yet
> - Receiver is responsible for marking `read: true` after successful processing
> - **Failure scenario**: If the receiver crashes before marking as read, the message is rediscovered on the next poll — this is **at-least-once delivery semantics**
>
> In contrast, protocol messages like `permission_request` and `shutdown_request` don't need read marks because they are routed by `markMessagesAsReadByPredicate` using type — **the type itself is the routing key**, no need to care about individual message read status. This forms an interesting asymmetry: regular text messages use read marks (consumer-driven), protocol messages use type routing (system-driven).

Key code (`writeToMailbox` lines 173-176):

```
const newMessage: TeammateMessage = {
  ...message,
  read: false,  // Fixed as false on write
}
messages.push(newMessage)
```

Key Function Design: Type Guards

Each protocol message type has a corresponding `isXxx()` type guard:

```
// teammateMailbox.ts, lines 435-447
export function isIdleNotification(
  messageText: string,
): IdleNotificationMessage | null {
  try {
    const parsed = jsonParse(messageText)
    if (parsed && parsed.type === 'idle_notification') {
      return parsed as IdleNotificationMessage
    }
  } catch {
    // Not JSON or not a valid idle notification
  }
```

```
  return null
}
```

Why return `null` instead of `boolean`? This allows the caller to simultaneously get both "is it type X" and "if so, what is the value" — reducing duplicate queries.

> **Note**
>
> Protocol message types are covered in the next section.

`isStructuredProtocolMessage`: The Boundary Between LLM Context and Protocol Processing

This is one of the most critical design decisions in the entire message system (`teammateMailbox.ts` lines 1073-1095). It is essentially a **message semantic routing table**:

Routing Target	Message Type	Processing Method	Delivery Semantics
System Processing (Deterministic)	permission*request/response, shutdown_request, mode_set_request, plan_approval**	Code routes, not injected into LLM	At-least-once
LLM Processing (Non-deterministic)	Regular text messages (idle_notification injected but without sensitive instructions)	Injected as context, AI decides independently	At-most-once (after read mark, won't appear again)
Not Processed (Filtered)	plan_approval_response, shutdown_approved	Code processes, not injected into LLM	—

> **Note**
>
> > **Delivery Semantics Design Choice**: Protocol messages use **at-least-once** delivery (`markRead()` is executed only after enqueuing to AppState). This means if the process crashes, the message is rediscovered and reprocessed on the next poll. This is especially critical for `shutdown_request` — if a shutdown request is missed, the teammate runs forever. But for `permission_request`, duplicate processing might cause duplicate popups — the solution is using `request_id` as an idempotency key (already implemented in the code), so even if messages are delivered multiple times, the processing result is idempotent.
> >
> > The source code comment (`teammateMailbox.ts` lines 1065-1072) reveals the full context of this decision: `getTeammateMailboxAttachments` is the LLM's context generator, called on every tool call. If a protocol message is consumed by it first, it appears as raw text in the LLM's attachments — the LLM might ignore it, rewrite it, or respond in an unexpected way. **Correct handling of these messages requires precise program state changes, and cannot rely on the LLM's non-deterministic reasoning**.

```
// Code comment explicitly states:
// "These message types have specific handlers in useInboxPoller that route them
// to the correct queues (workerPermissions, workerSandboxPermissions, etc.).
// If getTeammateMailboxAttachments consumes them first, they get bundled as
// raw text in attachments and never reach their intended handlers."
export function isStructuredProtocolMessage(messageText: string): boolean {
  return (
```

```
        type === 'permission_request' ||        // 1. Permission request
        type === 'permission_response' ||       // 2. Permission response
        type === 'sandbox_permission_request' || // 3. Sandbox network permission request
        type === 'sandbox_permission_response' || // 4. Sandbox network permission response
        type === 'shutdown_request' ||          // 5. Shutdown request
        type === 'shutdown_approved' ||         // 6. Shutdown approved
        type === 'team_permission_update' ||    // 7. Team permission update
        type === 'mode_set_request' ||          // 8. Mode change request
        type === 'plan_approval_request' ||     // 9. Plan approval request
        type === 'plan_approval_response'       // 10. Plan approval response
  )
}
```

Problem Scenario (Race Condition):

```
1. useInboxPoller reads Mailbox file → finds permission_request
2. Almost simultaneously, getTeammateMailboxAttachments (LLM attachment generation) also
reads the same file
3. Both try to mark the message as read
4. If the LLM-side function marks first, the message changes from "unread" to "read"
5. When useInboxPoller polls again, it thinks there are no new messages → permission_request
is never processed
```

Solution: Messages for which `isStructuredProtocolMessage` returns `true` are **not sent as LLM attachments**, processed only by `useInboxPoller`.

> **Deeper Architectural Insight**: This design solves a **dual-consumer routing problem**. In traditional queue systems, each message is processed by only one consumer; but Mailbox is essentially broadcast-type (both consumers read the same file). `isStructuredProtocolMessage` acts as the routing strategy: structured protocol messages go to system processing (change runtime state), regular text messages go to LLM processing (generate natural language responses). The design principle of this boundary is: **any operation involving state changes (permissions, shutdown, mode changes) should not go through the LLM** — because the LLM's response is non-deterministic and cannot drive a precise protocol state machine.
>
> The comment at the source location (`teammateMailbox.ts` lines 1065-1081) explicitly states:
>
> > "If getTeammateMailboxAttachments consumes them first, they get bundled as raw text in attachments and never reach their intended handlers."
> > This shows that this routing problem was not an afterthought — there was foresight from the initial design of protocol message types: **once a protocol message is injected into the LLM context, there's no guarantee it can be correctly processed** — because the LLM might choose to ignore it, rewrite it, or respond in an unexpected way. Therefore, the best strategy is fundamentally preventing them from entering that channel.

Note

- useInboxPoller: Team Lead's React UI polls the Mailbox file once per second, routing protocol messages (permission requests, shutdown requests, etc.) to code processing (popup approval, state changes, etc.) to change system state. LLM doesn't participate; after reading messages:

```
  permission_request          → Popup for user approval → User decides (not LLM)
  sandbox_permission_request  → Popup for user approval → User decides (not LLM)
  permission_response         → Code updates callback state → Not injected into LLM
  sandbox_permission_response → Code updates callback state → Not injected into LLM
  team_permission_update      → Code updates permission context → Not injected into LLM
  mode_set_request           → Code updates permission mode → Not injected into LLM
  plan_approval_request      → Code auto-approves → Re-injects into LLM context (tells
  LLM "teammate just submitted a plan")
  shutdown_request           → Code does nothing → Injects into LLM context (tells LLM
"need to shutdown")
  shutdown_approved          → Code cleans up member → Re-injects into LLM context (tells
LLM "X has shut down")
```

- getTeammateMailboxAttachments: Called before LLM sends messages, reads regular text messages from Mailbox file, injected as Prompt context. Code doesn't participate; AI understands and decides how to respond independently.

`formatTeammateMessages`: XML-Wrapped LLM Prompt Injection

```
// teammateMailbox.ts, lines 373-389
export function formatTeammateMessages(messages: Array<{
  from: string; text: string; timestamp: string; color?: string; summary?: string
}>): string {
  return messages.map(m => {
    const colorAttr = m.color ? ` color="${m.color}"` : ''
    const summaryAttr = m.summary ? ` summary="${m.summary}"` : ''
    return `<${TEAMMATE_MESSAGE_TAG}
teammate_id="${m.from}"${colorAttr}${summaryAttr}>\n${m.text}\n</${TEAMMATE_MESSAGE_TAG}>`
  }).join('\n\n')
}
```

Output format:

```
<teammate_message teammate_id="researcher" color="#4ECDC4" summary="Research results
completed">
This is the completion report for the research task...
</teammate_message>
```

`color` and `summary` are embedded as XML attributes, while `text` is treated as a CDATA segment. This has two benefits:

1. **High information density**: Attributes can be directly recognized by the LLM in the prompt (color for speaker identification, summary for quick overview)
2. **Prevents XML injection**: Nested XML tags are treated as plain text, won't break the outer structure

`getLastPeerDMSummary`: Extracting Summary from Message History

`teammateMailbox.ts` lines 1149-1183, design in team communication: **When sending a message, the SendMessage tool call at the end of that round is treated as that round's behavioral summary**

```
// Traverse message history, searching backward from the end
for (let i = messages.length - 1; i >= 0; i--) {
  const msg = messages[i]

  // Stop when encountering a user message (string content) — this is a new round
  if (msg.type === 'user' && typeof msg.message.content === 'string') {
    break
  }

  // Found SendMessage tool call, extract summary
  if (block.type === 'tool_use' && block.name === SEND_MESSAGE_TOOL_NAME) {
    return `[to ${to}] ${summary}`
  }
}
```

Design Intent: If an Agent round ends with sending a message to a peer, that sending behavior is treated as the summary of that round. This is used in `idle_notification` to tell Team Lead: "This Teammate sent what message to whom in round N."

> **Deeper Semantic Insight**: The assumption behind this design is: **the most important behavior in an Agent round is its output to peers**. This is the **Message as Deliverable** pattern in distributed systems — in multi-agent systems, message passing between agents is itself the carrier of work output, not a byproduct.
>
> The detail of traversing forward until encountering a `user` message defines an important **work round boundary**: user messages (whether from Team Lead instructions or other sources) represent the start of a new round. This means a "unit of work" is all reasoning and actions from after the previous user message to before the current user message. **The appearance of SendMessage within this interval signals that the Agent believes the work is complete and ready to communicate with peers.**
>
> If a Teammate sends messages to multiple peers in one reasoning round, only the last one is included in the summary — this is an information volume tradeoff: more messages means a longer summary, higher cost for UI display. Choosing the last one means "the newest" and "the most relevant" (because the Agent's conclusions are more mature toward the end of reasoning).

Complete Message Lifecycle

```
[Sender Agent decides to send message]
    │
    ▼
writeToMailbox()
    │ JSON.stringify(protocol message) → text field
    │ read = false (fixed value)
    │ color = sender's color
    ▼
~/.claude/teams/{team}/inboxes/{recipient}.json
    │
    ▼
[Recipient polling: useInboxPoller / waitForNextPromptOrShutdown]
    │
    ├─ isStructuredProtocolMessage() = true?
    │   │
```

```
    │   ├─ Yes (permission/shutdown/mode, etc.):
    │   │   → Route to dedicated queue (not as LLM context)
    │   │   → markMessagesAsReadByPredicate (preserve protocol message marks)
    │   │
    │   └─ No (regular text message):
    │       → formatTeammateMessages() → XML
    │       → Inject into LLM context as attachment
    │       → markMessagesAsRead() (mark all as read)
```

Message Deduplication: Dual-Source Race Handling

`getTeammateMailboxAttachments` needs to pull teammate messages from two places:

1. **File Mailbox**: Messages arrive between two polls (every tool call)
2. **AppState.inbox**: useInboxPoller temporarily stores messages here mid-turn execution, submitted after turn ends (1s polling)

But there are races:

```
1. getTeammateMailboxAttachments reads file → finds message M
2. useInboxPoller also reads same file → puts M in AppState.inbox
3. getTeammateMailboxAttachments reads AppState again → M is read twice
```

Solutions:

1. Cross-source deduplication (from|timestamp|text.slice(0,100) key) — prevents messages from appearing in both file mailbox and AppState.inbox simultaneously
   - Using first 100 characters instead of full text because when text is long, the first 100 characters are sufficient to uniquely identify a message, while avoiding large text concatenation overhead
2. Idle notification folding (only keep latest idle per agent) — prevents repeated idle reads across multiple polls
   - `idle_notification`'s `idleReason` has three values: `available` (idle, can take tasks), `interrupted` (interrupted), `failed` (failed). Team Lead only needs to know "what state this member is currently in", the latest idle notification always has more information than older ones
   - If multiple state changes occur within a very short time window (< poll interval), only the last one is perceived by Team Lead. This information loss is **intentional**: at 500ms polling granularity, high-frequency state jumps are invisible to Team Lead, which simultaneously avoids UI flickering and LLM context noise
3. Structured message filtering (isStructuredProtocolMessage) — prevents two consumers from simultaneously routing protocol messages
   - Key design: markRead() is executed only after the message is successfully enqueued to AppState, not marked as read in the file beforehand. This way, if the process crashes, the message is rediscovered on the next poll and won't be silently discarded.

3.6 SwarmPermissionRequest —— Permission Request (pending/resolved directories)

Definition: `src/utils/swarm/permissionSync.ts` (928 lines)

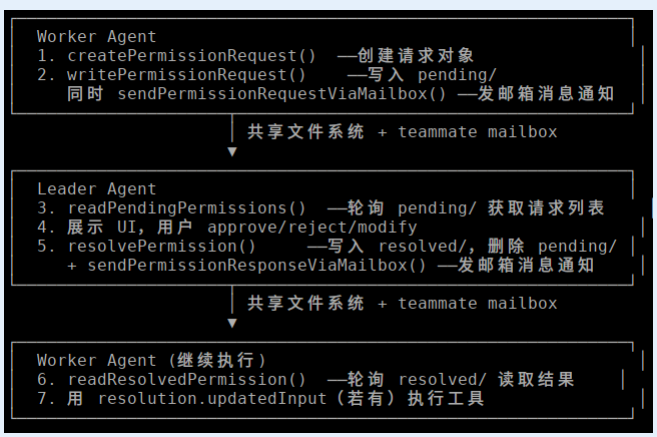
```
export const SwarmPermissionRequestSchema = z.object({
  id: z.string(),                  // Format: "perm-{timestamp}-{random}"
  workerId: z.string(),             // Requesting agentId
```

```
  workerName: z.string(),          // Requesting name
  workerColor: z.string().optional(),
  teamName: z.string(),
  toolName: z.string(),            // Tool name (e.g., "Bash")
  toolUseId: z.string(),           // Tool call ID (for tracking)
  description: z.string(),         // Natural language description
  input: z.record(z.string(), z.unknown()),
  permissionSuggestions: z.array(z.unknown()),
  status: z.enum(['pending', 'approved', 'rejected']),
  resolvedBy: z.enum(['worker', 'leader']).optional(),
  resolvedAt: z.number().optional(),
  feedback: z.string().optional(),
  updatedInput: z.record(z.string(), z.unknown()).optional(),
  permissionUpdates: z.array(z.unknown()).optional(),
  createdAt: z.number(),
})

export type PermissionResolution = {
  decision: 'approved' | 'rejected'
  resolvedBy: 'worker' | 'leader'
  feedback?: string
  updatedInput?: Record<string, unknown>
  permissionUpdates?: PermissionUpdate[]
}
```

Note

- This is the permission coordination layer in the Swarm multi-agent collaboration framework
- When a Worker Agent encounters an operation requiring user authorization (e.g., executing Bash commands, writing files) during tool call execution, it cannot directly pop up user interaction prompts, because users only interact with the system at the Leader Agent end
- Therefore, Workers need to forward permission requests to the Leader, who displays the UI and collects the user's decision on their behalf, then returns the result to the Worker



Key Implementation Details

- createPermissionRequest (L167): Automatically reads workerId/workerName/workerColor from environment variables, no need for caller to pass manually.
- writePermissionRequest (L215): Uses directory-level .lock file + lockfile.lock() for atomic writes, prevents concurrent writes from corrupting files.

- resolvePermission (L360): Reads pending/ → Updates fields (status, resolvedBy, etc.) → Writes to resolved/ → Deletes original file in pending/ (both steps done under the same lock, ensuring atomicity).
- cleanupOldResolutions (L452): Periodically cleans up the resolved/ directory, default 1 hour retention, prevents file accumulation.
- sendPermissionResponseViaMailbox (L734): In addition to the filesystem's resolved/ path, Leader also actively pushes messages to Worker via teammate mailbox — this is a more timely notification mechanism (alternative to Worker's pure polling).

Self-Approval Mechanism (resolvedBy: 'worker')

The `resolvedBy` field supports the 'worker' value, indicating the system supports Worker-side self-approval. This is used in scenarios:

- User has previously set "**always allow**" rules for similar operations
- Worker finds it has sufficient permissions, approves directly, doesn't disturb the user
  In this case, Worker writes to resolved/ without needing to notify Leader and can continue execution on its own.

3.7 TeamTask —— Shared Task Entry

TeamTask is a **persistent, cross-process, DAG-dependent task collaboration system** serving Claude Code's multi-agent team (Swarm/Team) scenario. In a team:

- Leader creates a task list, assigns tasks to Teammates by priority or dependency
- Each Teammate claims tasks and marks them completed after finishing
- Any agent can read and operate on the same task list, forming true collaboration

> Swarm and Team are not two separate systems, but two perspectives of the same infrastructure:
> Swarm = Runtime infrastructure for multi-agent collaboration (how to run, communicate, coordinate)
> Team = Organizational unit of collaboration (who works with whom, who is the leader)
> In other words: Swarm is the "operating system", Team is the "process group"

	Swarm	Team
本质	多智能体运行时框架	框架中的一个组织单元
实现位置	swarm/ 整个目录	swarm/teamHelpers.ts（一份 config.json）
核心文件	teammateMailbox.ts、spawnInProcess.ts	teamHelpers.ts 中的 TeamFile
谁更基础	**更底层**，提供通信 + 生命周期 + 协调	**构建在 Swarm 之上**，提供成员列表和领导关系
关系	–	Team 使用 Swarm 的所有能力来实现协作

Definition: `src/utils/tasks.ts` (863 lines)

```
export const TaskSchema = z.object({
  id: z.string(),
  subject: z.string(),              // Task title
  description: z.string(),          // Detailed description
  activeForm: z.string().optional(), // Progressive form (for UI spinner)

  // Ownership
  owner: z.string().optional(),      // Current claimer agentId

  // Status
  status: z.enum(['pending', 'in_progress', 'completed']),
```

```
  // Dependency (DAG support)
  blocks: z.array(z.string()),     // Tasks this task blocks
  blockedBy: z.array(z.string()),   // Tasks this task depends on

  // Metadata
  metadata: z.record(z.string(), z.unknown()).optional(),
})

export type ClaimTaskResult = {
  success: boolean
  reason?:
    | 'task_not_found'      // Task doesn't exist
    | 'already_claimed'      // Already claimed
    | 'already_resolved'     // Already completed
    | 'blocked'              // Blocked by other tasks
    | 'agent_busy'           // Agent is busy
  task?: Task
  busyWithTasks?: string[]   // List of in-progress tasks
  blockedByTasks?: string[]  // Tasks blocking this task
}
```

---

4. Team Creation and Management

4.1 TeamCreateTool Complete Flow

Definition: `src/tools/TeamCreateTool/TeamCreateTool.ts` (241 lines)

```
const inputSchema = z.strictObject({
  team_name: z.string()
    .describe('Name for the new team to create.'),
  description: z.string().optional()
    .describe('Team description/purpose.'),
  agent_type: z.string().optional()
    .describe('Type/role of the team lead (e.g., "researcher", "test-runner").'),
})
```

Detailed Creation Steps:

```
Step 1: Pre-check
  ├─ Check if already in a team (nested teams prohibited)
  └─ If team_name conflicts, auto-generate unique name (add numeric suffix)

> **Idempotent Creation Design Analysis**: The auto-numbering strategy for team_name
conflicts (team-1, team-2...) is a form of **Idempotent Creation**. If the user calls
`TeamCreateTool` twice with the same team_name, the second call doesn't fail but creates a
new team. This is reasonable in CLI interaction — users aren't penalized for misoperations.
>
> General rule: If "duplicate execution" cost < "error + interruption" cost → use idempotent
design

Step 2: Build TeamFile object
```

```
  const teamFile: TeamFile = {
    name: finalTeamName,
    description: _description,
    createdAt: Date.now(),
    leadAgentId: formatAgentId(TEAM_LEAD_NAME, finalTeamName),
    leadSessionId: getSessionId(),
    members: [{
      agentId: leadAgentId,
      name: TEAM_LEAD_NAME,
      agentType: leadAgentType,
      model: leadModel,
      joinedAt: Date.now(),
      tmuxPaneId: '',
      cwd: getCwd(),
      subscriptions: [],
    }],
  }

Step 3: Write TeamFile → ~/.claude/teams/{team}/config.json
Step 4: Register session cleanup hook → auto-cleanup on CLI exit
Step 5: Initialize task list directory → ~/.claude/tasks/{sanitized}/
Step 6: Set leader team context
Step 7: Update AppState with teamContext and teammates
```

4.2 Adding Team Members

Definition: `src/utils/swarm/addMemberToTeam.ts`

```
async function addMemberToTeam(params: {
  teamName: string; name: string; agentType?: string; model?: string;
  color?: string; planModeRequired?: boolean; backendType: BackendType;
  paneId?: string; sessionId?: string
}): Promise<TeammateIdentity> {
  const teamFile = await readTeamFileAsync(teamName)
  const existing = teamFile.members.find(m => m.name === params.name)
  if (existing) return toTeammateIdentity(existing)

  teamFile.members.push({
    agentId: formatAgentId(params.name, teamName),
    name: params.name, agentType: params.agentType, model: params.model,
    color: params.color ?? assignTeammateColor(params.name),
    planModeRequired: params.planModeRequired, joinedAt: Date.now(),
    tmuxPaneId: params.paneId ?? '', cwd: getCwd(),
    subscriptions: [], backendType: params.backendType, isActive: true,
  })

  await writeTeamFileAsync(teamName, teamFile)
  setAppState(prev => ({ ...prev, teamContext: { ...prev.teamContext,
    teammates: { ...prev.teamContext.teammates, [newMember.agentId]: newMember } } }))
  return toTeammateIdentity(newMember)
}
```

Note

Notable design points:

1. TeamFile write has no file lock — concurrent spawns may cause silent overwrites
2. Dual writes are non-atomic — teamFile (disk) and AppState.teamContext (memory) may become permanently inconsistent
3. In-process vs. out-of-process teammate prompt passing paths differ — mixing causes duplicate messages
4. Implicit leader self-registration — leader automatically writes itself to TeamFile

4.3 Team Destruction (Shutdown Protocol)

Definition: `src/tools/TeamDeleteTool/TeamDeleteTool.ts`

```
// Destruction flow:
// 1. Send ShutdownRequest to all members (via mailbox)
// 2. Wait for ShutdownApproved responses (max 30s normal, skip if force)
// 3. Force kill unresponsive tmux panes
// 4. Cleanup filesystem → Delete ~/.claude/teams/{team}/
// 5. Unregister team context from AppState
// 6. Unregister session cleanup hook
```

---

5. Agent Lifecycle Management

5.1 Lifecycle State Machine

```
[Creation] → [registerTask] → [LocalAgentTask / InProcessTeammateTask]
             → [In-Proc Runner / TmuxBackend / ITermBackend]
             → [runAsyncAgentLifecycle] → [Normal Complete / abort()]
             → [finalizeAgentTool] → [enqueueAgentNotification] → [Cleanup]
```

5.2 runAsyncAgentLifecycle Complete Implementation

Definition: `src/tools/AgentTool/agentToolUtils.ts`

```
export async function runAsyncAgentLifecycle({
  taskId, abortController, makeStream, metadata, description,
  toolUseContext, rootSetAppState, agentIdForCleanup,
  enableSummarization, getWorktreeResult,
}): Promise<void> {
  const tracker = createProgressTracker()
  const resolveActivityDescription = createActivityDescriptionResolver(toolUseContext,
metadata.agentType)

  for await (const message of makeStream(undefined)) {
    if (abortController.signal.aborted) {
      await killAsyncAgent(taskId, rootSetAppState); return
    }
    updateProgressFromMessage(tracker, message, resolveActivityDescription,
toolUseContext.tools)
    appendMessageToLocalAgent(taskId, message, rootSetAppState)
    const progress = getProgressUpdate(tracker)
    updateLocalAgentTaskProgress(taskId, progress, rootSetAppState)
```

```
  }

  const result = finalizeAgentTool(agentMessages, agentId, metadata)
  await completeAsyncAgent(taskId, result, rootSetAppState)
  clearInvokedSkillsForAgent(agentIdForCleanup)
  clearDumpState()
}
```

---

6. In-Process Execution Engine

6.1 Core Configuration

Definition: `src/utils/swarm/inProcessRunner.ts` (1552 lines)

```
export type InProcessRunnerConfig = {
  identity: TeammateIdentity; taskId: string; prompt: string
  agentDefinition?: CustomAgentDefinition; teammateContext: TeammateContext
  toolUseContext: ToolUseContext; abortController: AbortController
  model?: string; systemPrompt?: string; systemPromptMode?: 'default' | 'replace' | 'append'
  allowedTools?: string[]; allowPermissionPrompts?: boolean
  description?: string; invokingRequestId?: string
}
```

6.2 Complete Message Loop

```
const POLL_INTERVAL_MS = 500

export async function runInProcessTeammate(config: InProcessRunnerConfig):
Promise<InProcessRunnerResult> {
  const canUseTool = createInProcessCanUseTool(config.identity, config.abortController, ...)

  while (!config.abortController.signal.aborted) {
    const waitResult = await waitForNextPromptOrShutdown(config.identity,
config.abortController, ...)

    if (waitResult.type === 'shutdown_request') { await handleShutdown(config); return {
success: true, messages: [] } }
    if (waitResult.type === 'pending_message') { await
executeUserMessage(waitResult.message, config); continue }
    if (waitResult.type === 'leader_message') { await
handleLeaderMessage(waitResult.message, config); continue }
    if (waitResult.type === 'peer_message') { await handlePeerMessage(waitResult.message,
config); continue }
    if (waitResult.type === 'task_assignment') { await executeTask(waitResult.task, config);
continue }
  }

  await sendIdleNotification(config.identity.agentName, color, config.identity.teamName, {
idleReason: 'interrupted' })
  return { success: true, messages: [] }
}
```

6.2.1 Deep Analysis: Dual-Layer AbortController Design

Problem: In traditional Agent frameworks, "stop current work" and "kill Agent" are the same operation.

```
const abortController = new AbortController()           // Lifecycle (kill)
const currentWorkAbortController = createAbortController() // Work-level (interrupt only)
```

User presses Escape: Only triggers `currentWorkAbortController`, teammate enters idle state, waiting for new tasks.
Team Lead sends shutdown request: Triggers top-level `abortController`, teammate exits completely.

> This design also implies a **member identity persistence** promise: when user presses Escape, the teammate doesn't disappear, and messages previously sent by Team Lead remain valid.

6.2.2 Deep Analysis: Content Replacement State Historical Memory

`inProcessRunner.ts` lines 1035-1042:

```
let teammateReplacementState = toolUseContext.contentReplacementState
  ? createContentReplacementState() : undefined
```

> "Without persisting state across iterations...wire prefix differs → cache miss."

Problem: Claude Code uses **Prompt Caching**. If each Agent call uses a brand new content replacement decision state, the compressed message prefix of call 1 and call 2 differ → API cache key invalidates → every request misses cache → costs double.

Solution: Share `contentReplacementState` across calls, ensuring **incremental and consistent** decisions.

> This reveals Claude Code's fine-grained control over **Prompt Caching cost optimization**. The wire prefix must be identical for cache to hit. This design has an interesting parallel with React's `useRef`: both use mutable persistent references to break the "every render is fresh data" assumption.

6.3 waitForNextPromptOrShutdown Priority Algorithm

Four-layer priority:

```
First:   In-memory pendingUserMessages (from SendMessageTool) — ultra-fast, no IO
Second:  Shutdown request — prevents member from never exiting
Third:   Team leader message — leader represents user intent
Fourth:  Peer messages (FIFO) — processed in time order
Fifth:   Auto task claiming — lowest priority, only when idle
```

6.4 Permission Bridge Implementation

```
function createInProcessCanUseTool(identity, abortController, onPermissionWaitMs?):
CanUseToolFn {
  return async (tool, input, toolUseContext, assistantMessage, toolUseID) => {
    // Path A: Tool in permission whitelist → return allow/deny directly
    const decision = hasPermissionsToUseTool(tool.name, input)
    if (decision !== 'ask') return decision
```

```
    // Path B: Bash + classifier auto-approval
    if (tool.name === 'Bash') {
      const result = await runClassifierAutoApproval(tool.name, input, toolUseContext)
      if (result !== 'defer') return result
    }

    // Path C: Leader UI approval (preferred)
    const leaderQueue = getLeaderToolUseConfirmQueue()
    if (leaderQueue) {
      return new Promise(resolve => {
        leaderQueue.add({
          tool, input, toolUseId: toolUseID, assistantMessage,
          workerBadge: { color: identity.color ?? '#888', agentName: identity.agentName },
          requestId: generateRequestId(),
          onAllow: (updatedInput, permissionUpdates) => {
            persistPermissionUpdates(permissionUpdates)
            applyPermissionUpdates(permissionUpdates)
            resolve('allow')
          },
          onReject: () => resolve('deny'),
          onModified: () => resolve('modified'),
        })
      })
    }

    // Path D: Mailbox fallback (500ms polling)
    return new Promise(resolve => {
      const requestId = generateRequestId()
      registerPermissionCallback({ requestId, toolUseId: toolUseID,
        onAllow: () => resolve('allow'), onDeny: () => resolve('deny'),
        onModified: (modifiedInput) => resolve('modified') })
      sendPermissionRequestViaMailbox({ id: requestId, workerId: identity.agentId,
        workerName: identity.agentName, workerColor: identity.color,
        teamName: identity.teamName, toolName: tool.name, toolUseId: toolUseID,
        description: summarizeToolUse(tool.name, input), input,
        permissionSuggestions: [], status: 'pending', createdAt: Date.now() })
      const pollInterval = setInterval(async () => {
        if (abortController.signal.aborted) { clearInterval(pollInterval); resolve('deny');
return }
        onPermissionWaitMs?.(Date.now() - startTime)
        const allMessages = await readMailbox(identity.agentName, identity.teamName)
        const response = allMessages.find(m => isPermissionResponse(m.text) &&
          parseMessage(m.text).request_id === requestId)
        if (response) { clearInterval(pollInterval);
resolve(parseMessage(response.text).subtype === 'success' ? 'allow' : 'deny') }
      }, 500)
    })
  }
}
```

6.4.1 Dual-Channel Permission Bridge and Anti-Escalation Mechanism

Dual-Channel Design

- **UI channel** (preferred): Real-time push, no polling delay
- **Mailbox channel** (fallback): Works even when UI is unavailable

Anti-Escalation: preserveMode

```
onAllow(updatedInput, permissionUpdates) {
  persistPermissionUpdates(permissionUpdates)
  const updatedContext = applyPermissionUpdates(currentAppState.toolPermissionContext,
permissionUpdates)
  setToolPermissionContext(updatedContext, { preserveMode: true }) // Critical!
}
```

Without `preserveMode`: Worker gains permission → Leader's permission context is polluted.
With `preserveMode`: Leader's `permissionMode` stays `auto`. `preserveMode: true` ensures **permissions gained by Workers don't overflow into Leader's global state**.

> Code comment: "prevent workers' transformed 'accepted' context from leaking back" — the problem isn't just `permissionMode` pollution, but **permission decision history** itself being inherited across trust boundaries.

Bash Auto-Classification (Three-Layer)

```
Tool call → Layer 1: Safe tool whitelist (~20 tools, 100% bypass)
           → Layer 2: Bash prefix rules (git status, ls -la, etc., no LLM)
           → Layer 3: YOLO LLM classifier (fallback)
```

---

7. Inter-Process Communication (IPC) —— Mailbox System

7.1 Mailbox File Format

Path: `~/.claude/teams/{team_name}/inboxes/{agent_name}.json`

```
{ "messages": [{ "from": "researcher", "text": "...", "timestamp": "ISO", "read": false,
"color": "#4ECDC4", "summary": "..." }] }
```

Design Philosophy:

1. Append-only log + read-free / write-lock → At-Least-Once delivery
2. Lock-free read, locked write → High concurrent read performance
3. Dual consumer + type filtering → getTeammateMailboxAttachments (LLM) and useInboxPoller (system) don't interfere
4. Plain text and JSON coexist in same array
5. Index-based mark-as-read (potential issue with concurrent readers)
6. File path as identity anchor → uses agent name, not UUID

7.2 File Lock Implementation

```
const LOCK_OPTIONS = { retries: { retries: 10, minTimeout: 5, maxTimeout: 100 } }

async function writeToMailbox(recipientName, message, teamName?) {
  const inboxPath = getInboxPath(recipientName, teamName)
```

```
  await ensureInboxDir(inboxPath)
  if (!fs.existsSync(inboxPath)) fs.writeFileSync(inboxPath, JSON.stringify({ messages: [],
version: 1 }))
  await properLockfile.lock(`${inboxPath}.lock`, LOCK_OPTIONS)
  try {
    const data = JSON.parse(fs.readFileSync(inboxPath, 'utf8'))
    data.messages.push({ ...message, read: false })
    fs.writeFileSync(inboxPath, JSON.stringify(data, null, 2))
  } finally { await properLockfile.unlock(`${inboxPath}.lock`) }
}
```

7.3 Message Reading Strategy

```
async function readMailbox(agentName, teamName?): Promise<TeammateMessage[]>
async function readUnreadMessages(agentName, teamName?): Promise<TeammateMessage[]>
async function markMessageAsReadByIndex(agentName, teamName, messageIndex): Promise<void> //
Uses lock
async function markMessagesAsRead(agentName, teamName?): Promise<void>
```

7.4 Message Serialization Format

All structured messages are stored as **JSON strings** in the `text` field:

- Send: Object → JSON.stringify → text
- Read: text → JSON.parse → Type guard (isIdleNotification, isPermissionRequest, etc.)

---

8. Permission Management System

8.1 Three-Layer Permission Model

```
User Permission (CLI startup) → intersect → Team Permission (User ∩ teamAllowedPaths) →
filter → Member Permission (actual available tools)
```

8.2 pending/resolved Directory Mechanism

Definition: `src/utils/swarm/permissionSync.ts`

```
~/.claude/teams/{team}/permissions/pending/{id}.json  ← Created by Worker, read by Leader
~/.claude/teams/{team}/permissions/resolved/{id}.json ← Written by Leader, read by Worker
```

Self-approval (resolvedBy: 'worker'): Used when user previously set "always allow" rules for similar operations.

8.3 Permission Cleanup Mechanism

```
export async function cleanupOldResolutions(teamName?, maxAgeMs = 3600000): Promise<number>
{
  // Read resolved/ directory → filter .json files → parallel check resolvedAt → delete if
expired
  // Returns deletion count for caller alerting
}
```

Design: resolvedAt || createdAt (fallback), delete on parse failure, >= not > boundary, passive triggering.

9. Execution Backend Abstraction Layer

9.1 PaneBackend Interface

Definition: `src/utils/swarm/backends/types.ts`

```
export type PaneBackend = {
  type: BackendType  // 'tmux' | 'iterm2' | 'in-process'
  displayName: string; supportsHideShow: boolean
  isAvailable(): Promise<boolean>; isRunningInside(): Promise<boolean>
  createTeammatePaneInSwarmView(name, color): Promise<CreatePaneResult>
  sendCommandToPane(paneId, command, useExternalSession?): Promise<void>
  setPaneBorderColor(paneId, color, useExternalSession?): Promise<void>
  setPaneTitle(paneId, name, color, useExternalSession?): Promise<void>
  enablePaneBorderStatus(windowTarget?, useExternalSession?): Promise<void>
  rebalancePanes(windowTarget, hasLeader): Promise<void>
  killPane(paneId, useExternalSession?): Promise<boolean>
  hidePane(paneId, useExternalSession?): Promise<boolean>
  showPane(paneId, targetWindowOrPane?, useExternalSession?): Promise<boolean>
}
```

Backend design distinctions:

后端	运行环境	pane 创建方式	hide/show	特点
tmux	任意环境	tmux split-window 或 new-window	通过 pane 移动到隐藏窗口实现	最通用，跨平台
iterm2	macOS + iTerm2	iTerm2 原生 split (AppleScript)	iTerm2 原生支持	体验最好，需安装 it2 CLI
in-process	任意环境	不涉及 pane，纯 AsyncLocalStorage 模拟	不支持 (supportsHideShow: false)	无外部依赖，适合无终端场景

9.2 Backend Auto-Detection Algorithm

```
isRunningInTmux() → yes → return tmux
isInITerm2() → yes + hasIt2Cli → return iterm2
                 → no it2Cli + isTmuxAvailable() → return tmux fallback
                 → no it2Cli + no tmux → return none
isTmuxAvailable() → yes → return tmux
→ return none
```

9.3 Tmux Backend Key Commands

```
// Create pane: tmux split-window -h -l '70%' → send-keys teammate command → set title/color
// Hide pane: tmux break-pane -d -s {paneId} -t HIDDEN_SESSION_NAME
// Show pane: tmux join-pane -h -s HIDDEN_SESSION_NAME:{paneId} -t {window}
// Rebalance: main-vertical with leader fixed at 20%, tiled without leader
```

9.4 In-Process Backend

```
type = 'in-process', supportsHideShow = false
createTeammatePaneInSwarmView → returns { paneId: 'inprocess:{name}', sessionName:
'inprocess' }
sendCommandToPane → throws Error (uses Mailbox instead)
```

```
killPane → returns true (via abortController)
hidePane/showPane → returns false
```

---

10. Team Task List and Distributed Task Claiming

10.1 Task Directory Structure

```
~/.claude/tasks/{taskListId}/  (taskListId = sanitize(teamName))
├── {taskId}.json  ← Task file
├── .lock          ← File lock
└── .highwatermark ← Prevents ID collision
```

10.2 Task Claiming Algorithm (Optimistic Lock + File Lock)

```
async function claimTask(taskListId, taskId, claimantAgentId): Promise<ClaimTaskResult> {
  // Check 1 (no lock): Fast elimination of non-existent tasks
  if (!fs.existsSync(taskPath)) return { success: false, reason: 'task_not_found' }

  await properLockfile.lock(`${taskPath}.lock`, { retries: 30, minTimeout: 5, maxTimeout:
100 })
  try {
    // Check 2 (with lock): re-read and verify
    const task = JSON.parse(fs.readFileSync(taskPath))
    if (task.status === 'completed') return { success: false, reason: 'already_resolved' }
    if (task.owner && task.owner !== claimantAgentId) return { success: false, reason:
'already_claimed' }

    // DAG dependency check
    const blockedByTasks = await Promise.all(task.blockedBy.map(id => getTask(taskListId,
id)))
    const unresolved = blockedByTasks.filter(t => t?.status !== 'completed')
    if (unresolved.length > 0) return { success: false, reason: 'blocked', blockedByTasks:
unresolved.map(t => t!.id) }

    task.status = 'in_progress'; task.owner = claimantAgentId
    fs.writeFileSync(taskPath, JSON.stringify(task))
    return { success: true, task }
  } finally { await properLockfile.unlock(`${taskPath}.lock`) }
}
```

Check-Lock-Check Pattern: First layer without lock quickly filters non-existent tasks (common case), avoiding pointless lock contention. Most polls return directly, completely avoiding lock competition.

10.3 Auto-Claiming in In-Process Runner

```
async function tryClaimNextTask(taskListId, agentName): Promise<string | undefined> {
  const pending = (await listTasks(taskListId))
    .filter(t => t.status === 'pending' && !t.owner)
    .sort((a, b) => {
      const prioDiff = ((b.metadata?.priority as number) ?? 0) - ((a.metadata?.priority as
number) ?? 0)
```

```
      if (prioDiff !== 0) return prioDiff
      return ((a.metadata?.createdAt as number) ?? 0) - ((b.metadata?.createdAt as number)
?? 0)
    })
  if (pending.length === 0) return undefined
  const result = await claimTask(taskListId, pending[0].id, agentName)
  return result.success ? pending[0].id : undefined
}
```

10.2.1 DAG Dependency Detection: Two-Pass Implicit Topological Sort

```
function findAvailableTask(tasks): Task | undefined {
  const unresolvedTaskIds = new Set(tasks.filter(t => t.status !== 'completed').map(t =>
t.id))
  return tasks.find(task =>
    task.status === 'pending' && !task.owner &&
    task.blockedBy.every(id => !unresolvedTaskIds.has(id))
  )
}
```

- First pass: Build incomplete task ID set (O(n))
- Second pass: Check dependencies for each pending task (O(n × m))
- Complexity: O(n × m), sufficient for n=5-20, m<5
- Naturally supports **dynamic DAG**: new tasks can be added anytime, blockedBy chains auto-unlock on completion

---

11. Agent Toolchain and Sub-Agent Dispatch

11.1 AgentTool Parameter Schema

Definition: `src/tools/AgentTool/AgentTool.ts`

```
// Base: description, prompt, subagent_type, model override, run_in_background
// Multi-Agent: name (addressable via SendMessage), team_name, mode ('plan'/'bypass')
// Full: isolation ('worktree'/'remote' for Git worktree), cwd override
export const inputSchema = lazySchema(() => {
  const schema = feature('KAIROS') ? fullInputSchema() : fullInputSchema().omit({ cwd: true
})
  return isBackgroundTasksDisabled ? schema.omit({ run_in_background: true }) : schema
})
```

11.2 Tool Filter

```
export function filterToolsForAgent({ tools, isBuiltIn, isAsync, permissionMode }): Tools {
  return tools.filter(tool => {
    if (tool.name.startsWith('mcp__')) return true
    if (toolMatchesName(tool, EXIT_PLAN_MODE_V2_TOOL_NAME) && permissionMode === 'plan')
return true
    if (ALL_AGENT_DISALLOWED_TOOLS.has(tool.name)) return false
    if (!isBuiltIn && CUSTOM_AGENT_DISALLOWED_TOOLS.has(tool.name)) return false
    if (isAsync && !ASYNC_AGENT_ALLOWED_TOOLS.has(tool.name)) {
```

```
      if (isAgentSwarmsEnabled() && isInProcessTeammate()) {
        if (toolMatchesName(tool, AGENT_TOOL_NAME)) return true
        if (IN_PROCESS_TEAMMATE_ALLOWED_TOOLS.has(tool.name)) return true
      }
      return false
    }
    return true
  })
}
```

11.3 Agent Result Integration

```
export function finalizeAgentTool(agentMessages, agentId, metadata): AgentToolResult {
  const content = agentMessages
    .filter(m => m.role === 'assistant')
    .flatMap(m => (Array.isArray(m.content) ? m.content : []))
    .filter(b => b.type === 'text')
    .map(b => ({ type: 'text', text: b.text }))

  const totalToolUseCount = agentMessages.filter(m => m.role === 'user' && (m as any).type
=== 'tool_result').length
  const totalTokens = estimateTokens(agentMessages)

  return { agentId, agentType: metadata.agentType, content,
    totalToolUseCount, totalDurationMs: Date.now() - metadata.startTime,
    totalTokens, usage: extractUsage(agentMessages) }
}
```

---

12. Coordinator Mode

12.1 Coordinator Role Definition

Definition: `src/coordinator/coordinatorMode.ts`

A **specially configured Team Lead** with a system prompt instructing it on task decomposition and delegation.

Core responsibilities: Analyze requests → Decompose into parallel subtasks → Select members → Dispatch via AgentTool → Coordinate via SendMessageTool → Integrate results.

Workflow stages: Research (Workers, parallel) → Synthesis (Coordinator) → Implementation (Workers) → Verification (Workers)

> **Design Insight**: This is a **Macro-pipeline** architecture. It solves: (1) LLM attention bandwidth problem, (2) Producer-consumer decoupling, (3) Maximizing parallelism (limited by Amdahl's Law due to serial Synthesis + Verification).

12.2 Coordinator Task Decomposition Algorithm

```
function decomposeTask(userRequest): TaskPlan {
  if (analysis.complexity === 'low') return { mode: 'direct', plan: [...] }
  if (analysis.complexity === 'medium') return { mode: 'sequential', plan: [...] }
  // High complexity: research (parallel) → synthesis → implementation (parallel) →
verification
```

```
  return { mode: 'parallel_team', phases: [...] }
}
```

---

13. State Management (AppState)

13.1 Team-Related AppState Structure

```
type TeamContext = { teamName: string; teamFilePath: string; leadAgentId: string; teammates:
Record<string, TeammateInfo> }
type TeammateInfo = { name: string; agentType?: string; color: string; tmuxSessionName:
string; tmuxPaneId: string; cwd: string; spawnedAt: number }
type AppState = { tasks: Record<string, TaskState>; teammateMailboxes: Record<string,
TeammateMessage[]> }
```

13.2 Team Memory Synchronization

```
// File: ~/.claude/teams/{team}/memory/{entryId}.json
type TeamMemoryEntry = {
  id: string; type: 'user' | 'feedback' | 'project' | 'reference'
  name: string; description: string; content: string
  createdAt: number; updatedAt: number; createdBy: string
  visibility: 'team' | 'private'; version: number
}
```

13.3 Three-Layer Identity Recognition and Zero-Overhead Isolation

```
Layer 1 (Highest): AsyncLocalStorage — zero-overhead memory access
Layer 2: dynamicTeamContext — inter-process, shared via files
Layer 3: Environment variables — fallback for tmux-spawned child processes
```

Why AsyncLocalStorage over Web Worker: LLM inference is I/O-bound (waiting for API responses), not CPU-bound. The main isolation need is **logical context isolation**, not **physical resource isolation**.

Promise Chain Lock in TmuxBackend

```
let paneCreationLock: Promise<void> = Promise.resolve()
function acquirePaneCreationLock(): Promise<() => void> {
  const newLock = new Promise<void>(resolve => { release = resolve })
  const previousLock = paneCreationLock
  paneCreationLock = newLock
  return previousLock.then(() => release!)
}
```

Key: Doesn't block event loop, avoids deadlock in async/await with exceptions, `release()` called in try/finally regardless of success.

---

14. Key Constants Quick Reference

Constant	Value	File	Purpose
`POLL_INTERVAL_MS`	`500`	inProcessRunner.ts	Mailbox polling interval
`PERMISSION_POLL_INTERVAL_MS`	`500`	inProcessRunner.ts	Permission response polling
`TEAMMATE_MESSAGES_UI_CAP`	`50`	InProcessTeammateTask/types.ts	Message history cap
`PANE_SHELL_INIT_DELAY_MS`	`200`	TmuxBackend.ts	tmux pane startup delay
`LOCK_OPTIONS.retries`	`30`	tasks.ts	Task file lock retries
`LOCK_OPTIONS.retries`	`10`	teammateMailbox.ts / permissionSync.ts	Mailbox/permission retries
`cleanupOldResolutions.maxAgeMs`	`3600000`	permissionSync.ts	1-hour retention

---

15. Key Algorithm Detailed Flows

15.1 Teammate Complete Startup Flow

```
spawnInProcessTeammate() called
  → Generate AgentId ("researcher@my-team")
  → Create independent AbortController (not linked to parent)
  → Create AsyncLocalStorage context
  → Register InProcessTeammateTaskState to AppState
  → Asynchronously start runInProcessTeammate() (non-blocking)
  → Return spawn result; runInProcessTeammate runs independently
  → Enter message loop (POLL_INTERVAL_MS = 500ms)
    → shutdown_request? → Send ShutdownApproved, exit
    → Other messages → Process, continue loop
```

15.2 Permission Request-Response Complete Sequence

```
Teammate: canUseTool('Bash') → Not in whitelist
  → Generate requestId
  → Path A: leaderQueue.add() → UI popup → User approves → onAllow() → resolve('allow')
  → Path B: writePermissionRequest() → poll mailbox (500ms) → detect response → resolve
  → Execute Bash / throw rejection
```

15.3 Shared Task Claiming Sequence

```
Member A: claimTask('task-1') → lock → read → pending, owner=null → write
owner='reviewer@team' → unlock → {success: false, already_claimed}
Member B: claimTask('task-2') → lock → read → pending, owner=null → write
owner='researcher@team' → unlock → {success: true}
```

---

16. Summary and Design Evaluation

16.1 Core Design Highlights

Design	Implementation	Evaluation
AsyncLocalStorage Isolation	In-Process members via AsyncLocalStorage	Lightweight, zero process overhead
File Lock Mailbox	proper-lockfile + JSON	Simple, no external deps, throughput limited
Permission Bridge	Sub-Agent → Leader UI forwarding	Good UX, adds latency
Backend Abstraction	Unified PaneBackend interface	Clear, extensible
DAG Task Dependencies	blockedBy / blocks	Complex workflow support
Graceful Shutdown	ShutdownRequest → ShutdownApproved	Bidirectional confirmation
Message Priority	Shutdown > Leader > Peer > Task claim	Prevents starvation

16.2 Architectural Trade-offs

Aspect	Current	Alternative	Reason
IPC	File lock mailbox (polling)	Unix Socket / Redis	Node.js fs constraints
Permission	Sync wait (Promise blocking)	Async queue	Real-time first
Task distribution	Shared file + file lock	Dedicated queue	Offline-first
Member isolation	AsyncLocalStorage	Web Worker	Performance first
Member discovery	Filesystem	Service registry	Simplicity first

Claude Code Agent Team targets: "single developer, local machine, short-duration tasks" → 10 concurrent agents fully within file lock capacity.

16.3 Main Limitations

1. **Polling efficiency**: 500ms fixed polling → unnecessary IO at low volume, latency ceiling at high volume
2. **Single bottleneck**: Team Lead handles permission + messages + integration simultaneously
3. **No cross-machine**: Requires same filesystem path
4. **Permission serialization**: One member's request blocks same-group members
5. **No deadlock detection**: No timeout when members mutually wait
6. **Unbounded Mailbox**: No message count limit (unlike `TEAMMATE_MESSAGES_UI_CAP` which only limits AppState memory)

16.4 Deep Design Philosophy

Philosophy One: Pragmatic "Good Enough" Architecture

File polling is sufficient for single-machine scenarios; PaneBackend interface reserves extension points for advanced capabilities.

Philosophy Two: Dual-Layer AbortController UX

Escape → Teammate enters idle (doesn't disappear) → Team Lead can resend messages → Multi-round collaboration preserved.

Philosophy Three: Promise Chain Lock

```
return previousLock.then(() => release!) // Exception-safe: release() called in try/finally
```

This is the **Chained Call Pattern** in async scenarios — forms a natural ordered pipeline without blocking the event loop.

Philosophy Four: preserveMode (Principle of Least Privilege)

Worker permission context must not pollute Leader's global state. Both `permissionMode` AND **permission decision history** must not cross trust boundaries.

Philosophy Five: Eventual Consistency Over Strong Consistency

Lock-free read (for concurrency) + locked write (for append atomicity). Accepts brief inconsistency; pragmatic choice for local-first scenarios.

Philosophy Six: Graceful Degradation

Dual-channel permission: UI (preferred, real-time) + Mailbox (fallback, polling). System continues through Mailbox even if UI fails.

Philosophy Seven: Priority Queue Prevents Deadlock

Shutdown requests have highest priority → Prevents message flooding attacks from starving Team Lead's ability to terminate members.

16.5 Extension Suggestions

```
Short-term: Mailbox GC, WebSocket direct connection, member heartbeat detection
Mid-term: Redis Pub/Sub (cross-machine), task priority preemption, task timeout
Long-term: Service registry, dynamic load balancing, direct inter-member communication,
multi-level coordination
```

16.6 Horizontal Comparison with Mainstream Multi-Agent Frameworks

Feature	Claude Code Agent Team	AutoGen	LangGraph
IPC	File lock + JSON	gRPC / HTTP	In-memory
Permission	Hierarchical bridging	Independent Agent	Graph state
Task deps	DAG file	Programming-defined	Graph edges
Isolation	AsyncLocalStorage	Process/container	Thread/process/container
Cross-machine	Not supported	Supported	Supported
Execution	tmux/iTerm2/In-Process	Docker/process	Custom nodes
Graceful shutdown	ShutdownRequest bidirectional	terminate()	No built-in
Context compression	Prompt Caching + replacement state	None	None

Key Insight: Claude Code Agent Team is **deeply optimized for CLI scenarios**. Other frameworks pursue generality at the cost of complexity. `preserveMode`, Promise chain lock, dual-layer AbortController are rare elsewhere, embodying Claude Code's unique engineering trade-offs.

> **Common theme across "rare designs" — Cross-Entity Trust Boundary Management**:

Design	Boundary Problem	Solution
`preserveMode`	Worker → Leader permission history leakage	Write-time filtering
Dual-Layer AbortController	User interrupt ↔ System termination	Lifecycle/work separation
`isStructuredProtocolMessage`	System ↔ LLM message routing conflict	Type-based routing

Design	Boundary Problem	Solution
Priority queue	Peer → shutdown starvation	Shutdown jumps queue
`contentReplacementState`	Cache key inconsistency	Incremental decisions

These designs share a common essence: **constraining "things that shouldn't cross boundaries" within boundaries through technical means**. Running multiple Agents with different trust levels simultaneously in one process, this boundary management capability is the core of system stability.